Step-by-Step Tutorial for an Automated GF180MCU Digital Design Workflow

Maryam Shahangian and James E. Stine
Oklahoma State University
Electrical and Computer Engineering Department
Stillwater, OK 74078 USA
{maryam.shahangian, james.stine}@okstate.edu

1 Introduction

The goal of this tutorial is to provide a simple, step-by-step workflow for running OpenROAD inside a containerized environment. The tutorial first introduces the basic concept of a container and explains why containerized environments are useful for digital design workflows. A container provides a ready-to-use environment in which the required design tools are already installed and configured. As a result, users do not need to manually install tools or resolve software dependency issues.

The tutorial then demonstrates how OpenROAD can automate the major stages of a digital design flow. Rather than executing each stage manually, OpenROAD-flow-scripts can guide the design from RTL input through synthesis, place-and-route, and final GDSII layout generation.

To demonstrate the workflow, the tutorial first uses an example design that is already included within OpenROAD. This provides a complete example of the design flow inside the container, beginning with the RTL design files and ending with the generated GDSII layout. Finally, the tutorial explains how the same flow can be adapted to use the **OSU GF180** standard-cell library.

2 Understanding the Environment

In this tutorial, the design environment is provided through a containerized setup. This means that the chip-design tools are not installed directly on the host operating system. Instead, they are provided inside a prepared environment that can be launched consistently across different machines. In practical terms, the host computer remains the user's normal system, while the actual design workflow runs inside a preconfigured software environment containing the required tools, libraries, scripts, and dependencies.

2.1 What IIC-OSIC-TOOLS Is

IIC-OSIC-TOOLS is the containerized tool environment used in this tutorial. It provides an integrated, ready-to-use platform for IC design, with the required tools already bundled together. This makes setup easier and avoids the need to install and configure each tool manually.

2.2 What Docker Does

Docker is the software used to launch and manage the prepared environment. In simple terms, the container is the workspace that already contains the design tools, while Docker is the software that creates, starts, and manages that workspace. In this tutorial, Docker is used to run the IIC-OSIC-TOOLS container so that the OpenROAD workflow can be executed in a consistent and reproducible environment.

3 Installing Docker

Docker Desktop installers are available for multiple operating systems and architectures. The official Docker download page provides installers for Windows, Linux, and macOS. In this tutorial, the documented setup was performed on Windows. Therefore, the following steps focus specifically on the Windows installation path and the continuation of the environment setup.

1. Download Docker Desktop from the official Docker website for the appropriate operating system.

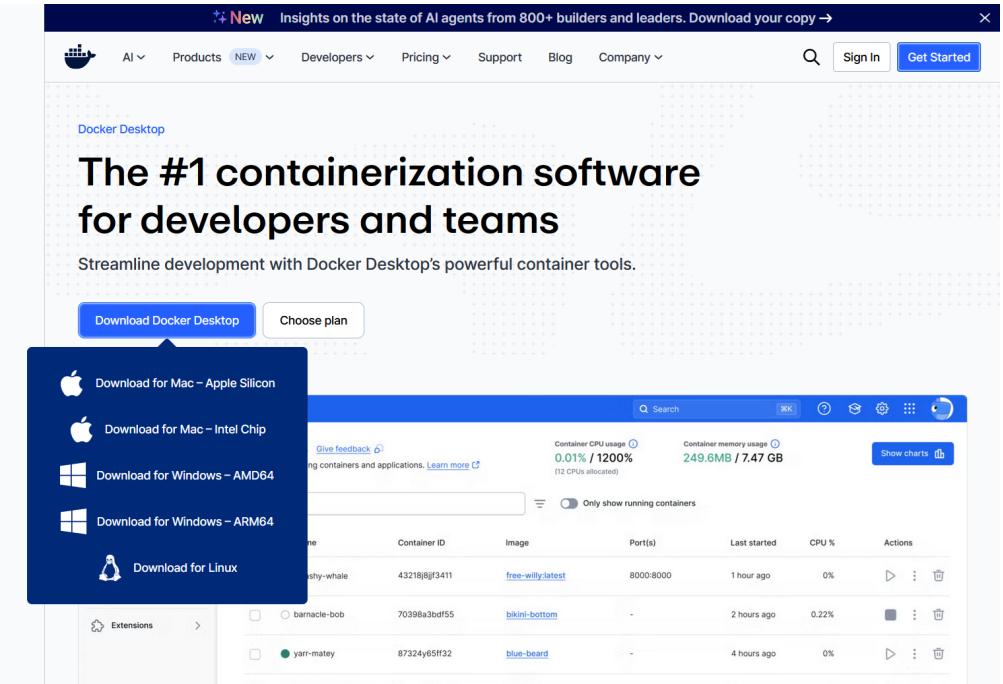

Figure 1: Docker Desktop download page for selecting the appropriate installer.

2. Follow the installation instructions provided by Docker Desktop to complete the installation on the host machine.

After Docker Desktop is installed, select the preferred method for accessing the container environment. Three access modes are available:

- **VNC Mode:** Provides a full graphical desktop environment that can be accessed through a web browser.
- **Jupyter Mode:** Provides a web-based notebook environment useful for running commands, documenting steps, and working interactively.

- **Shell Mode:** Provides a simple command-line terminal for running scripts and server-side tasks.

For this tutorial, Jupyter Mode is selected as the primary access method.

3.1 Select the Correct Chipathon Branch and Docker Image

After cloning the repository,

1. open Command Prompt (CMD) and run the following commands:

```
cd %USERPROFILE%\eda\designs\iic-osic-tools
```

```
git checkout ieee_chipathon
```

   This switches the repository to the branch used for the Chipathon setup.

2. Next, set the Docker image tag and start Jupyter:

```
SET DOCKER_TAG=chipathon26
start_jupyter.bat
```

   These commands switch to the Chipathon branch and start the Jupyter container using the Chipathon 26 image.

3. If a Jupyter container was already created previously using an older image, remove it first and then start it again:

```
docker rm -f iic-osic-tools_jupyter
SET DOCKER_TAG=chipathon26
start_jupyter.bat
```

4. After that, open Docker Desktop. Click the Play button next to `iic-osic-tools_jupyter` and wait a few seconds for the container to start. Once the container is running, go to the **Ports** column and click `8888:8888` to open the Jupyter environment in a web browser.

Tip: If the port number shown in Docker Desktop is not clickable, manually open a web browser and go to: `http://localhost:8888`

For VNC mode, click the corresponding VNC port link in Docker Desktop, or manually open: `http://localhost:80` When prompted, enter the default password: `abc123` At this point, the JupyterLab interface should become visible in the browser, indicating that the container environment has started successfully and is ready for interactive use. The primary shared working directories are:

- Windows: `C:\Users\<username>\eda\designs`
- Container: `/foss/designs`

Use this folder for all files and designs throughout the tutorial.

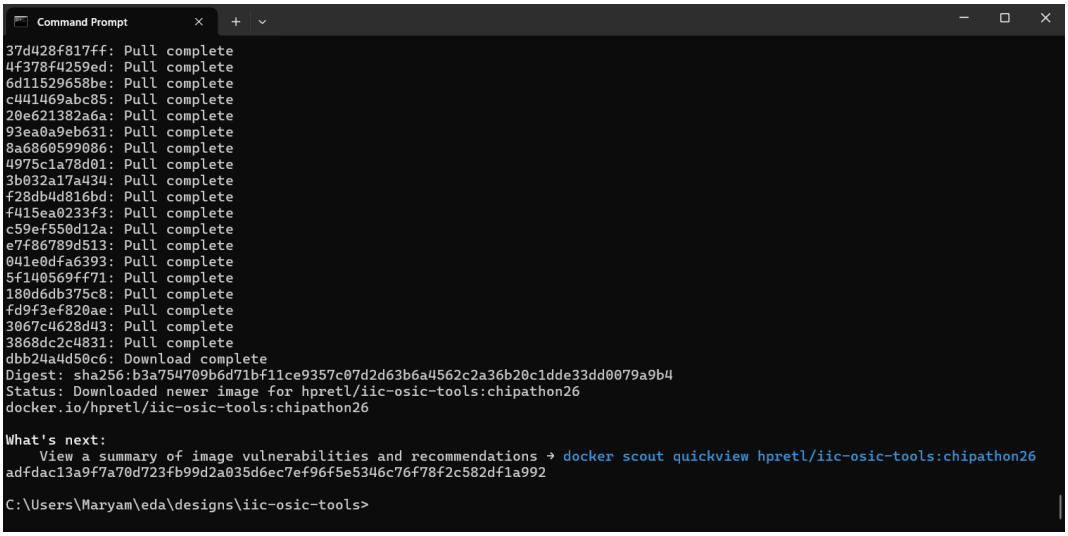

Figure 2: Successful startup of the Chipathon 26 Jupyter container in Command Prompt.

3.2 Check the Installed Tools

To open a terminal in JupyterLab, go to the **Launcher** page and, under **Other**, click **Terminal**. Then run the following commands:

```
cd /foss/designs && pwd && ls
```

To verify the installed tools and display the ORFS commit version, run:

```
yosys -V
```

```
openroad -version
```

```
echo $TOOLS
```

```
cat $TOOLS/openroad/ORFS_COMMIT
```

These commands confirm that the required tools are available inside the container and display the matching ORFS commit used by the environment.

What Is an "Image"?

In Docker terminology, an image is a prebuilt software environment that already contains the required tools and dependencies. In this tutorial, the **Chipathon26** image is used. This image is distributed through the IIC-OSIC-TOOLS environment and maintained by Harald Pretl and collaborators. It already includes the main tools required for the workflow, such as OpenROAD and Yosys.

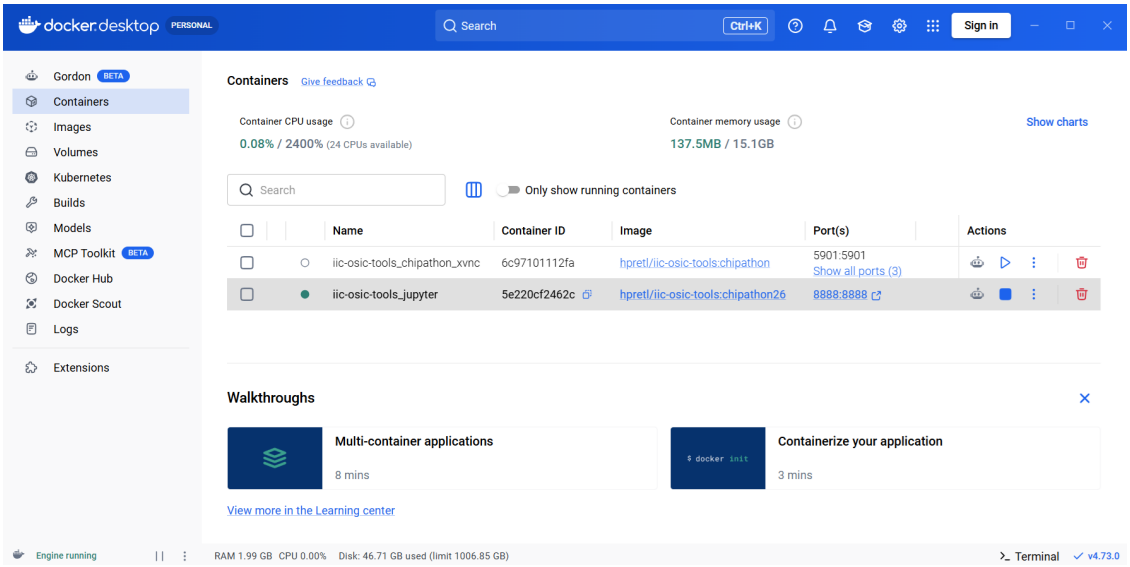

Figure 3: Docker Desktop showing the running Jupyter container and the available port link.

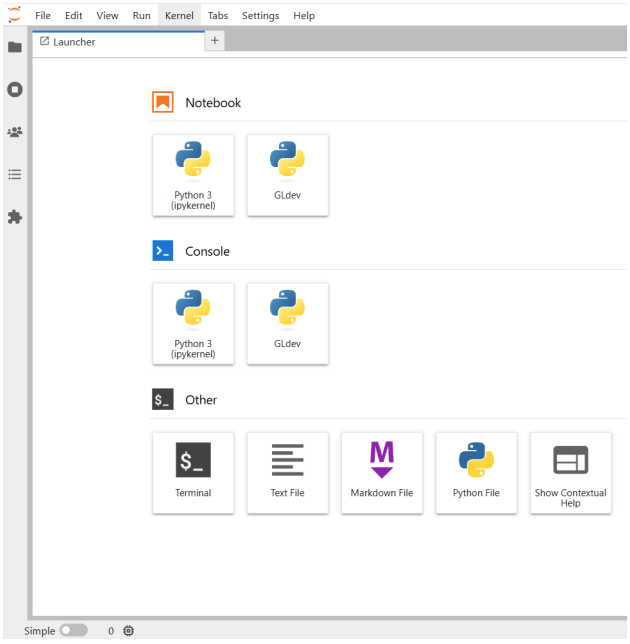

Figure 4: JupyterLab launcher page after the container starts successfully.

Why is the commit important?

The ORFS commit helps confirm that the OpenROAD-flow-scripts version matches the OpenROAD tools installed inside the container. This avoids compatibility problems caused by version mismatches.

3.3 Clone OpenROAD-flow-scripts

Next, clone the **OpenROAD-flow-scripts** repository and check out the commit version stored in the container image.

```
cd /foss/designs
```

```
git clone https://github.com/The-OpenROAD-Project/OpenROAD-flow-scripts.git
```

```
cd OpenROAD-flow-scripts
```

```
git checkout $(cat $TOOLS/openroad/ORFS_COMMIT)
```

This ensures that the flow scripts match the OpenROAD tools provided by the container image.

3.3.1 Setting the Tool Paths

Now configure the tool paths so that the flow uses the OpenROAD and Yosys executables installed inside the container.

```
cat >> ~/.bashrc <<'EOF'
export YOSYS_EXE=/foss/tools/yosys/bin/yosys
export OPENROAD_EXE=/foss/tools/openroad/bin/openroad
export OPENSTA_EXE=/foss/tools/openroad/bin/sta
EOF
```

```
source ~/.bashrc
```

This configuration step only needs to be performed once. Afterward, the terminal session will automatically recognize these tool paths.

Note: In this setup, the flow scripts expect the executables to exist under the local `tools/install` directory. However, the Chipathon 26 container already provides these tools under `/foss/tools`.

The following one-time setup creates symbolic links to the installed tools so that the OpenROAD flow can start normally.

```
cd /foss/designs/OpenROAD-flow-scripts
mkdir -p tools/install
ln -sfn /foss/tools/yosys tools/install/yosys
```

```
ln -sfn /foss/tools/openroad tools/install/OpenROAD
```

3.3.2 Run the Flow

Move to the flow directory and run:

```
cd /foss/designs/OpenROAD-flow-scripts/flow
```

```
make
```

This starts the OpenROAD flow using the tool versions provided by the current Chipathon 26 container image. At this stage, the OpenROAD environment inside the IIC-OSIC-TOOLS container is fully configured and ready for use. The environment is now prepared so that the focus can shift to the main objective: running and understanding chip-design examples.

4 Run the AES Example Design

In this section, the OpenROAD flow is tested using a ready-to-use example design provided by OpenROAD. This helps verify that the environment is functioning correctly before moving to custom designs. The AES example is used here. The goal is not to modify the design, but to understand how the automated flow operates from RTL input to final layout generation. The RTL source files for the example are located inside the **designs/src** directory. The platform-specific configuration files for AES are stored under the corresponding GF180 design directory. First, move to the designs directory and enter the source folder:

```
cd /foss/designs/OpenROAD-flow-scripts/flow/designs
cd src
ls
```

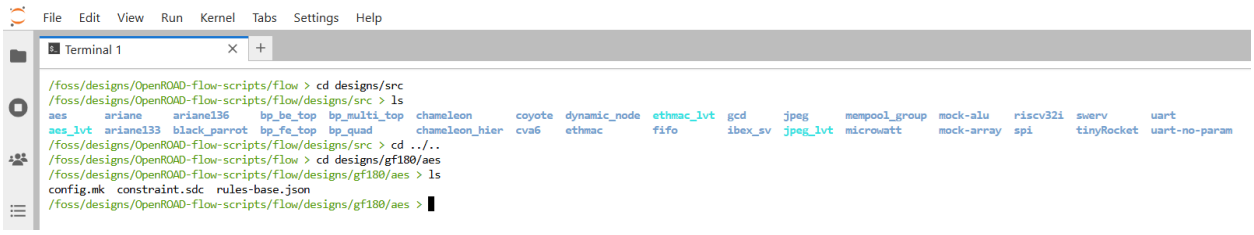

Figure 5: Listing available example designs in OpenROAD-flow-scripts, including the AES design example.

In this directory, the available example designs can be seen. One of them is **aes**. Next, move to the AES platform directory and list its files:

```
cd /foss/designs/OpenROAD-flow-scripts/flow/designs/gf180/aes
ls
```

The main configuration files for the AES example can now be seen, including:

- `config.mk`
- `constraint.sdc`

The **config.mk** file defines the primary design setup, while the **constraint.sdc** file contains the timing constraints used by the flow. Before running the AES example, it is useful to briefly inspect the **config.mk** file. This file contains the main configuration for the design, including the design name, target platform, Verilog source files, and timing constraints used during the flow.

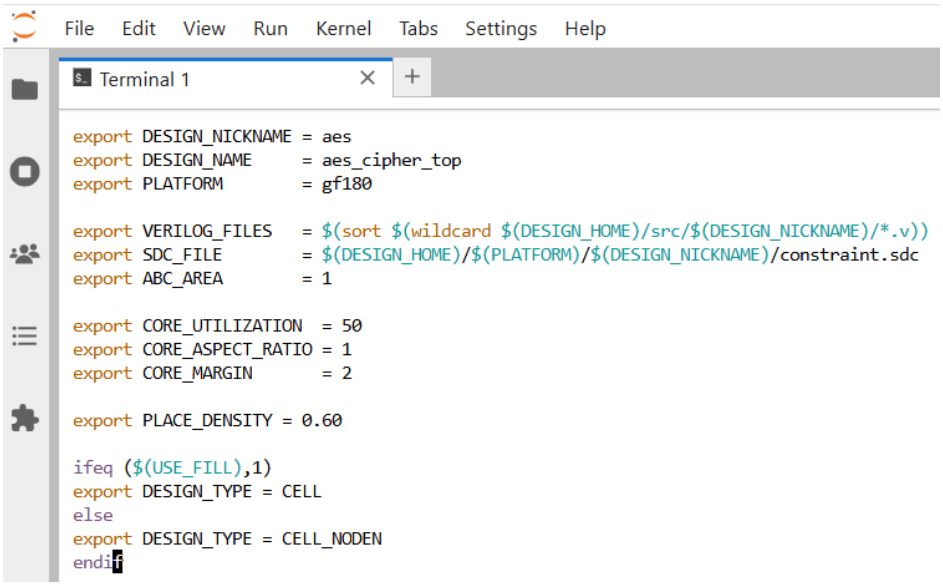

Figure 6: Example contents of the AES `config.mk` file used to define the design, platform, RTL sources, and implementation settings.

After that, move to the flow directory and run the AES example:

```
cd /foss/designs/OpenROAD-flow-scripts/flow
```

```
make DESIGN_CONFIG=./designs/gf180/aes/config.mk
```

As soon as this command is executed, the automated OpenROAD design flow begins. The first major stage is **synthesis**, where the RTL description is converted into a gate-level netlist. This step is performed by **Yosys**. After synthesis, the flow automatically continues through the remaining physical-design stages, including:

- Floorplanning
- Placement
- Clock Tree Synthesis (CTS)
- Routing

These stages are executed sequentially without requiring manual intervention. Depending on the complexity of the design and the available system resources, the complete flow may take several minutes to finish.

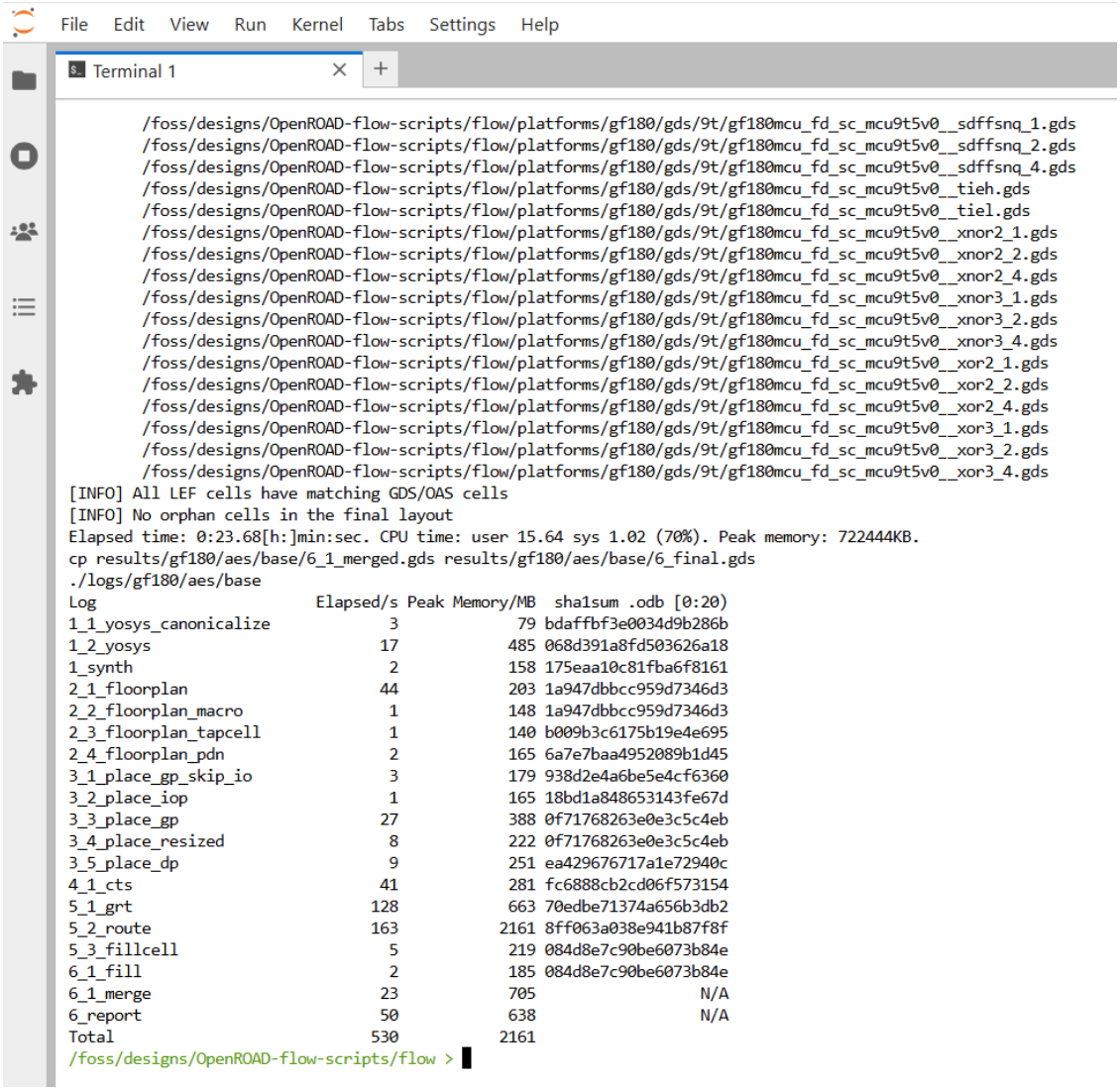

Figure 7: Successful completion of the automated OpenROAD AES implementation flow, including synthesis, placement, CTS, routing, and final GDS generation.

In Jupyter mode, the graphical interface may not appear because Jupyter provides a terminal-oriented environment rather than a full desktop interface. To view the final layout graphically, either use the VNC mode or open the generated GDS file locally using KLayout. A simple way to access the final layout is to download the generated GDS file directly from the JupyterLab file browser. The generated GDS file is typically located at:

```
/foss/designs/OpenROAD-flow-scripts/flow/results/gf180/aes/base/6_1_merged.gds
```

After downloading the file to the local computer, open it using **KLayout**.

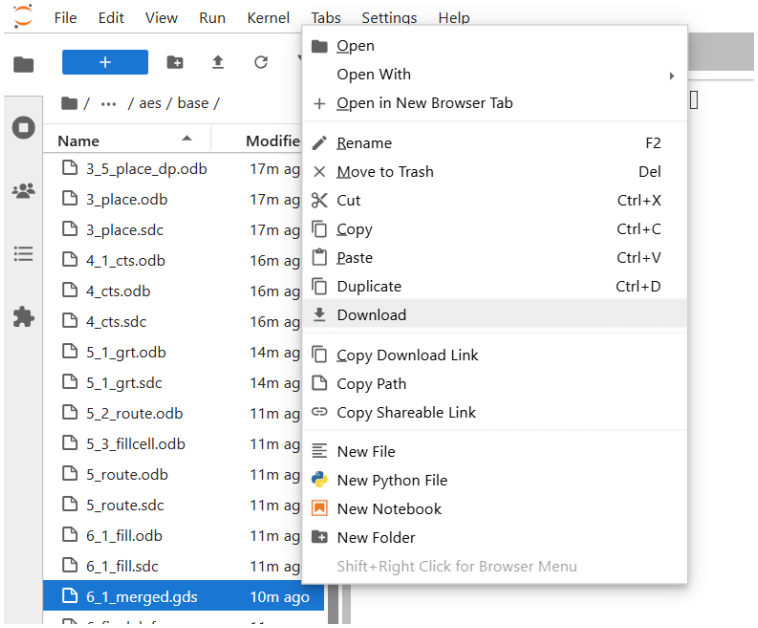

Figure 8: Downloading the generated merged GDS file from JupyterLab and opening it locally with KLayout.

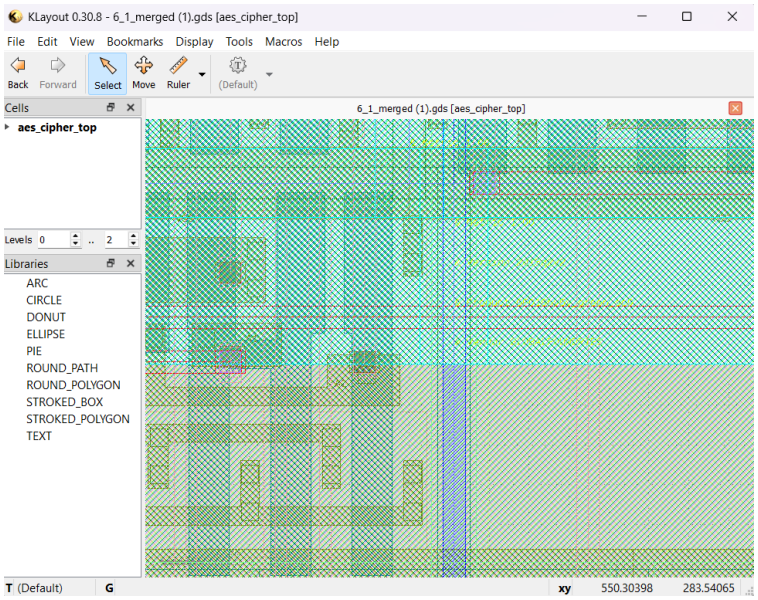

Figure 9: Viewing the final AES GDS layout in KLayout after completing the automated OpenROAD flow.

5 Worked Example: AES Design with the GF180MCU OSU Library

This section presents a worked example based on an AES design integrated with the GF180MCU OSU standard-cell library. The objective of this example is to demonstrate how a realistic RTL design can be prepared and executed using the automated OpenROAD implementation flow. Compared with a very small test example, the AES design provides a more meaningful demonstration because it produces realistic synthesis logs, intermediate outputs, and physical-design results that can be analyzed throughout the flow.

5.1 Design Objective and Source Setup

The main objective of this example was to synthesize and implement an AES design using the GF180MCU OSU standard-cell library together with the automated OpenROAD flow. This example was selected because it provides a more realistic workflow than a minimal test design and makes it possible to observe real outputs, logs, and stage-by-stage behavior throughout the implementation process. The first practical step was identifying the correct location for the AES RTL source files inside the OpenROAD-flow-scripts directory structure. After reviewing the organization of the source tree, a dedicated source directory for the AES design was created under the design hierarchy. The AES RTL files were then placed inside that directory so they could be referenced directly by the design configuration files. This step was important because the OpenROAD flow depends on a consistent directory structure in order to recognize and process the design correctly.

5.2 Design Configuration

After placing the AES source files, the next step was creating the design-specific configuration file for the AES project. The configuration file was created at:

```
~/OpenROAD-flow-scripts/flow/designs/gf180/aes/config.mk
```

The purpose of this file was to define the AES design within the automated flow and connect it to the appropriate source files, target platform, and implementation settings. This configuration step was necessary because the AES project needed to be treated as a custom design rather than one of the default examples already included in the repository. In practice, this file served as the primary entry point for launching the automated flow on the AES design.